Support vector machines with non-linear kernels

Fraida Fund

Contents

Kernel SVMs . . . 2
- Review: SVM prediction . . . 2
- Extension to non-linear decision boundary . . . 2
- SVM with basis function transformation . . . 2
- Example (1) . . . 2
- Example (2) . . . 2
- Example (3) . . . 3
- Example (4) . . . 3
- Kernel trick . . . 3
- Kernel as a similarity measure . . . 3
- Linear kernel . . . 4
- Polynomial kernel . . . 4

Using infinite-dimension feature space . . . 5
- Radial basis function kernel . . . 5
- RBF kernel: intuition . . . 5
- Feature mapping vs kernel . . . 5
- Kernel: not just for SVM . . . 5

Summary: SVM . . . 6
- Key expression . . . 6
- Key ideas . . . 6

Kernel SVMs

Review: SVM prediction

Given a set of support vectors S and associated α for each,

$$z = w_0 + \sum_{i \in S} \alpha_i y_i \langle \mathbf{x}_i, \mathbf{x}_t \rangle$$
$$\hat{y} = \text{sign}(z)$$

Measures inner product (a kind of "correlation") between new sample and each support vector.

For the geometric intuition/why inner product measures the similarity between two vectors, watch: 3Blue1Brown series S1 E9: Dot products and duality.

This SVM assumes a linear decision boundary. (The expression for z gives the equation of the hyperplane that separates the classes.)

Extension to non-linear decision boundary

- For logistic regression: we used basis functions of $\mathbf{x}$ to transform the feature space and classify data with non-linear decision boundary.
- Could use similar approach here?

SVM with basis function transformation

Given a set of support vectors S and associated α for each,

$$z = w_0 + \sum_{i \in S} \alpha_i y_i \langle \boldsymbol{\phi}(\mathbf{x}_i), \boldsymbol{\phi}(\mathbf{x}_t) \rangle$$
$$\hat{y} = \text{sign}(z)$$

Note: the output of $\boldsymbol{\phi}(\mathbf{x})$ is a vector that may or may not have the same dimensions as $\mathbf{x}$.

Example (1)

Suppose we are given a dataset of feature-label pairs in $\mathbb{R}^1$:

$$(-1, -1), (0, -1), (1, -1), (-3, +1), (-2, +1), (3, +1)$$

This data is not linearly separable.

Example (2)

Now suppose we map from $\mathbb{R}^1$ to $\mathbb{R}^2$ using $\boldsymbol{\phi}(x) = (x, x^2)$:

$$((-1, 1) - 1), ((0, 0), -1), ((1, 1), -1),$$
$$((-3, 9) + 1), ((-2, 4) + 1), ((3, 9) + 1)$$

This data *is* linearly separable in $\mathbb{R}^2$.

Example (3)

Suppose we compute $\langle \boldsymbol{\phi}(x_i), \boldsymbol{\phi}(x_t) \rangle$ directly:

- compute $\boldsymbol{\phi}(x_i)$
- compute $\boldsymbol{\phi}(x_t)$
- take inner product

How many operations (exponentiation, multiplication, division, addition, subtraction) are needed?

For each computation of $\langle \boldsymbol{\phi}(x_i), \boldsymbol{\phi}(x_t) \rangle$, we need five operations:

- (one square) find $\phi(x_i) = (x_i, x_i^2)$
- (one square) find $\phi(x_t) = (x_t, x_t^2)$
- (two multiplications, one sum) find $\langle \phi(x_i), \phi(x_t) \rangle = x_i x_t + x_i^2 x_t^2)$

Example (4)

What if we express $\langle \boldsymbol{\phi}(x_i), \boldsymbol{\phi}(x_t) \rangle$ as

$$K(x_i, x_t) = x_i x_t (1 + x_i x_t)$$

How many operations (exponentiation, multiplication, division, addition, subtraction) are needed to compute this equivalent expression?

Each computation of $K(x_i, x_t)$ requires three operations:

- (one multiplication) compute $x_i x_t$)
- (one sum) compute $1 + x_i x_t$
- (one multiplication) compute $x_i x_t (1 + x_i x_t)$

Kernel trick

- Suppose kernel $K(\mathbf{x}_i, \mathbf{x}_t)$ computes inner product in transformed feature space $\langle \boldsymbol{\phi}(\mathbf{x}_i), \boldsymbol{\phi}(\mathbf{x}_t) \rangle$
- For the SVM:

$$z = w_0 + \sum_{i \in S} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}_t)$$

- We don't need to explicitly compute $\boldsymbol{\phi}(\mathbf{x})$ if computing $K(\mathbf{x}_i, \mathbf{x}_t)$ is more efficient

Note that the expression we use to find the α_i values also only depends on the inner product, so the kernel works there as well.

Kernel as a similarity measure

- $K(\mathbf{x}_i, \mathbf{x}_t)$ measures "similarity" between training sample $\mathbf{x}_i$ and new sample $\mathbf{x}_t$
- Large K, more similarity; K close to zero, not much similarity
- $z = w_0 + \sum_{i=1}^{N} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}_t)$ gives more weight to support vectors that are similar to new sample - those support vectors' labels "count" more toward the label of the new sample.

Linear kernel

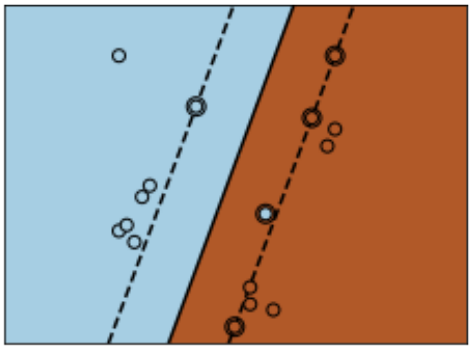


Figure 1: Linear kernel: $K(x_i, x_t) = x_i^T x_t$

Polynomial kernel

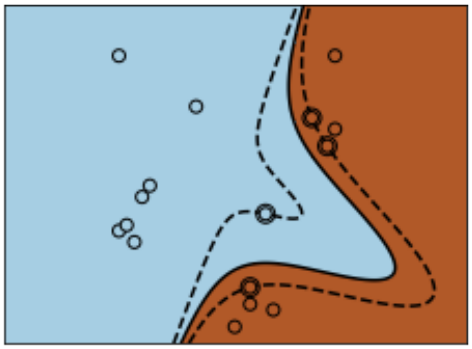


Figure 2: Polynomial kernel: $K(x_i, x_t) = (\gamma x_i^T x_t + c_0)^d$

Using infinite-dimension feature space

Radial basis function kernel

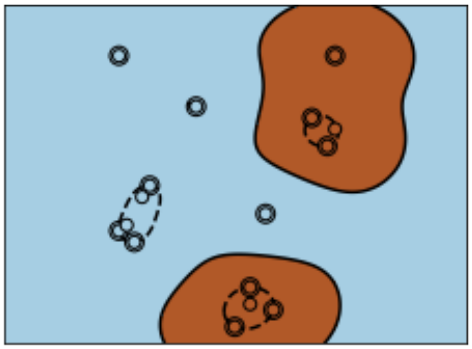


Figure 3: Radial basis function: $K(x_i, x_t) = \exp(-\gamma||x_i - x_t||^2)$. If $\gamma = \frac{1}{\sigma^2}$, this is known as the Gaussian kernel with variance σ^2.

RBF kernel: intuition

$$K_{\text{RBF}}(\mathbf{x}_i, \mathbf{x}_t) = \exp\left(-\gamma\|\mathbf{x}_i - \mathbf{x}_t\|^2\right)$$

- Similarity is 1 when the two samples are identical, and decays toward 0 as they get farther apart.
- Each support vector has a "bump" of influence around it; the decision for a new sample is a weighted vote of nearby support vectors.
- γ controls the width of the bumps:
  - large γ: narrow bumps, wiggly boundary, high variance
  - small γ: wide bumps, smooth boundary, high bias

This kernel corresponds to a feature mapping ϕ into an *infinite*-dimensional space! We could never compute $\phi(\mathbf{x})$ explicitly, but we can compute the kernel in $O(p)$ time. (The proof expands the exponential as an infinite sum of polynomial kernels; we won't go through it.)

Like KNN, RBF-kernel SVM makes predictions from nearby training samples, but it only needs to keep the support vectors, and each one has a learned weight α_i.

Feature mapping vs kernel

- **First approach**: basis function transformation AKA feature mapping
- **Current approach**: kernel - work in transformed space without explicit transformation
- **Next lesson**: wait and see!

A basis function transformation can be expensive if the dimensionality of the transformed feature space is large. With a kernel approach, we can work very efficiently in high dimensional feature space.

Kernel: not just for SVM

Kernels are used in other types of models -

- Kernel regression
- Gaussian process regression

Summary: SVM

Key expression

Decision boundary can be computed using an inexpensive kernel function on a small number of support vectors:

$$z = w_0 + \sum_{i \in S} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}_t)$$

($i \in S$ are the subset of training samples that are support vectors)

Key ideas

- Boundary with max separation between classes
- Tuning hyperparameters controls complexity
  - C for width of margin/number of support vectors
  - also kernel-specific hyperparameters
- Kernel trick allows efficient extension to higher-dimension space: non-linear decision boundary through transformation of features, but without explicitly computing high-dimensional features.